

EOS Worksheet#5 :

Inter-Process Communication (IPC)

Part 1 — 基本觀念

- Q1. 請解釋 semaphore 的種類、設計、用途與特徵 (註：mutex 須說明四大特徵) 。
- Q2. 為什麼 process 之間的溝通需要透過 kernel IPC 機制，不能只使用 global flag 嗎？
- Q3. 請解釋 message queue 跟 pipe 的差異 (設計、用途與特徵) 。
- Q4. 何謂 Priority Inversion？什麼情況下會出現這種問題？
- Q5. 請說明 barrier 跟 rendezvous synchronization。請提供 pseudo code。

Part 2 — 實作設計

- Q6. Linux Kernel 中 pipe 是一種以 rendezvous synchronization 來實現的 IPC 機制，常用於兩個 tasks 之間的資料交換。請 trace 並分析 pipe_read()、pipe_write() (fs/pipe.c)，說明 pipe 架構與 buffer, wait queues 之間的運作，特別說明其間 rendezvous 的運作方式。
- Q7. 呈上，參考 pipe.c，用 character device 實作一個 mini pipe，
 - a. 當 writer ready、reader not ready 時，writer waits
 - b. 當 reader ready、writer not ready 時，reader waits
 - c. 當雙方都 ready 時，喚醒對方並同步傳輸資料。
 - d. 你可以制定自己的 writing policy。

提示：writer 呼叫 write() 想寫資料，即為 writer ready。

提示：所謂 wait 可以用 wait_event_interruptible() 與在某個 queue 上等待條件成立。

提示：所謂喚醒，是指當狀態或條件改變時，使用 wake_up_interruptible() 喚醒對方。

提示：應該需要寫 read()、write()、poll() 等 function。

提示：可以使用固定大小的 buffer。

繳交：mini_pipe.c, Makefile, test code (cat, echo 或自己的測試程式) , test logs, README, diff (說明你的 mini_pipe 跟原來的 pipe.c 有什麼不同、你的 writing policy 。)

作業繳交說明

請於課堂結束前，將以下資料上傳至 E3：

- Word 檔案：包含前述問題的回答、執行結果截圖 (如果有的話) 、200 字心得
- 檔案名稱：ws5-學號.docx

註：使用 Gemini 或 ChatGPT 需一併附上你的提問問題。